

MODULE 34

State and Event Management in React

Learn how React manages state and events to build interactive, dynamic, and scalable user interfaces -- from useState fundamentals to controlled forms, lifted state, and effects.





WHY STATE MANAGEMENT MATTERS

State is the heart of every interactive application. Proper state management ensures your UI is predictable, responsive, and scalable.



Drives Interactivity

State holds data that changes over time. When state updates, React efficiently re-renders and the UI instantly reflects it.



Ensures Consistency

Good state management keeps data consistent across components and prevents conflicting or outdated information.



Enables Scalability

As applications grow, well-structured state makes it easier to add features and maintain the codebase.

PROBLEMS WITHOUT PROPER STATE MANAGEMENT



Unpredictable UI

UI may not reflect the latest data or user actions correctly.



Hard to Debug

Scattered state and unclear data flow make bugs difficult to trace.



Poor Maintainability

Complex, duplicated state logic leads to harder maintenance.



Performance Issues

Unnecessary re-renders and inefficient updates slow the app down.

STATE UPDATE FLOW IN REACT

1

User Action (Event)

2

State Updates

3

React Re-renders

4

UI is Updated

KEY TAKEAWAY State management is not just about storing data -- it's about managing change, flow, and synchronization across your application.



MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to build interactive, responsive, and maintainable user interfaces by mastering state and event management in React.

- **Understand React State** — explain what state is and how it drives dynamic changes in a component.
- **Work with Local State** — create and manage local component state using the useState Hook.
- **Manage the State Lifecycle** — understand how state updates trigger re-renders and how React manages mount, update, and unmount.
- **Use React Hooks** — understand the role of useState, useEffect, and other Hooks and when to use them in functional components.
- **Handle Events** — handle user events (onClick, onChange, onSubmit) effectively to build interactive experiences.
- **Manage Forms and Inputs** — manage controlled form state and input values using React state and event handling.
- **Implement Data Flow Patterns** — implement parent-to-child (props) and child-to-parent (callback) communication.
- **Understand the State Update Workflow** — explain how state updates flow through React and result in a UI re-render.
- **Apply Enterprise UI Patterns** — use state management techniques -- lifting state up, decomposition, derived data -- to build complex, real-world interfaces.
- **Follow Best Practices** — apply React state and event management best practices for clean, immutable, maintainable code.

KEY TAKEAWAY Mastering state and event management is essential for building dynamic, interactive, and scalable React applications that deliver a great user experience.

WHAT IS STATE?

State is a built-in object in React components that holds data which can change over time. When it changes, React automatically re-renders the component.



KEY CHARACTERISTICS

- **Dynamic** — state data can change during the lifetime of a component.
- **Local to Component** — state belongs to the component where it is declared.
- **Triggers Re-render** — updating state causes React to re-render the component.
- **Managed by React** — React manages state values and the component lifecycle.

EXAMPLE -- COUNTER COMPONENT

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

Holds Data

Changes Over Time

Triggers Re-render

Updates UI

KEY TAKEAWAY State is the heart of interactivity in React. It holds dynamic data and ensures your UI stays in sync with user actions and changes.

LOCAL COMPONENT STATE

Local component state is data managed within a single component, using React's useState Hook -- private to that component and isolated from others.



CHARACTERISTICS

- **Private** — state is accessible only within the component where it is declared.
- **Isolated** — changes in one component's state do not affect other components.
- **Triggers Re-render** — updating local state causes only that component to re-render.
- **Simple to Use** — ideal for UI-specific data like form inputs, toggles, and counters.

EXAMPLE -- ISOLATED COUNTER

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div className="counter">
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}

export default Counter;
```

Owned by Component

Private & Isolated

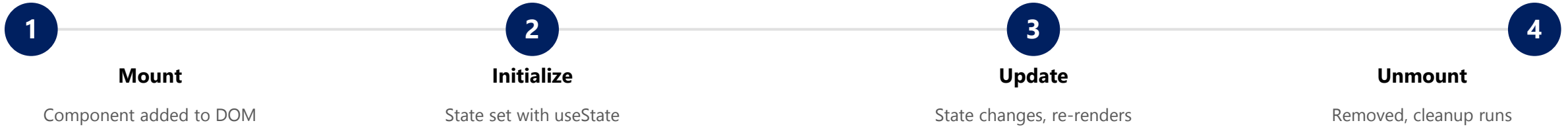
Triggers Re-render

Instant UI Updates

KEY TAKEAWAY Use local state for data that belongs to a single component and does not need to be shared with others.

STATE LIFECYCLE

The state lifecycle describes how state is created, updated, and destroyed during the life of a React component.



WHAT HAPPENS IN EACH PHASE

- **Mount** — the component is inserted into the DOM.
- **Initialize** — initial state is set using useState.
- **Update** — state updates cause the component to re-render with new values.
- **Unmount** — the component is removed from the DOM; cleanup logic (if any) runs.

EXAMPLE -- MOUNT / UNMOUNT WITH useEffect

```
function Demo() {
  const [count, setCount] = useState(0);
  useEffect(() => {
    console.log('Component Mounted');
    return () => {
      console.log('Component Unmounted');
    };
  }, []); // runs on mount and unmount
  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}
```

KEY TAKEAWAY Understanding the state lifecycle helps you write predictable, efficient, and bug-free React components.

INTRODUCTION TO REACT HOOKS

React Hooks are functions that let you use state and other React features without writing a class -- making components simpler and easier to understand.

WHY HOOKS?

- **Use React Features** — in functional components, without ever writing a class.
- **Reuse Logic** — share stateful behavior across components with custom Hooks.
- **Less Code** — cleaner and more readable than class-based lifecycle methods.
- **Better Organization** — related logic stays grouped together instead of split across lifecycle methods.

RULES OF HOOKS

- **Only call Hooks at the top level** — never inside loops, conditions, or nested functions.
- **Only call Hooks from React functions** — functional components or custom Hooks, not plain JavaScript functions.

BUILT-IN HOOKS YOU WILL USE

Hook	What It Does
useState	Adds state to a functional component.
useEffect	Performs side effects (fetch, subscriptions, timers) in a function component.
useContext	Accesses context values without a Consumer.
useRef	Stores a mutable value that does not cause a re-render.

HOOKS VS. CLASSES

Less boilerplate

Easier logic reuse

Simpler lifecycle (useEffect)

Easier to test

KEY TAKEAWAY Hooks let you use state and other React features in function components -- in a simpler, more flexible way than class components ever offered.

USESTATE HOOK

Syntax: `const [state, setState] = useState(initialState);` -- the most commonly used Hook in React, letting you add state to functional components.

HOW `useState` WORKS



INITIAL STATE CAN BE ANY TYPE

Type	Example
Number	<code>useState(0)</code>
String	<code>useState('Aman')</code>
Boolean	<code>useState(false)</code>
Object	<code>useState({ id: 1, name: 'Aman' })</code>
Array	<code>useState([])</code>

EXAMPLE

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div className="counter">
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

KEY POINTS

- State is local to the component where it is declared.
- Updating state does not change the value immediately in the current render.
- React batches state updates together for better performance.
- Never mutate state directly -- always call the setter function.

KEY TAKEAWAY `useState` is your go-to Hook for managing local state in functional components. It makes components simple, interactive, and easy to maintain.

USEEFFECT HOOK

useEffect lets you perform side effects in function components -- it runs after render and is used for data fetching, subscriptions, timers, and DOM updates.

HOW useEffect WORKS



COMMON DEPENDENCY PATTERNS

Deps	When It Runs	Use Case
[]	Once, on mount	Initial fetch, setup subscriptions/timers
[value]	When value changes	Re-fetch or re-sync on prop/state change
(none)	After every render	Rarely needed -- can cause performance issues

EXAMPLE -- DATA FETCHING WITH CLEANUP

```
useEffect(() => {
  let active = true;
  setLoading(true);
  fetch('/api/users')
    .then((res) => res.json())
    .then((data) => { if (active) setUsers(data); })
    .finally(() => setLoading(false));
  return () => { active = false; }; // cleanup
}, []); // empty array -> run once on mount
```

BEST PRACTICES

- Always specify dependencies
- Keep effects focused
- Clean up subscriptions/timers
- Avoid effects for derived data

KEY TAKEAWAY useEffect synchronizes your component with the outside world by running side effects at the right time -- always clean up what you start.



HANDLING USER EVENTS (1 OF 2)

React makes it easy to handle user interactions like clicks, key presses, form inputs, and more using event handlers.

WHAT IS AN EVENT?

An event is an action triggered by the user or the browser, such as clicking a button, typing in a field, or moving the mouse.

COMMON USER EVENTS IN REACT



HOW EVENT HANDLING WORKS



EXAMPLE -- BUTTON CLICK EVENT

```
import { useState } from 'react';

function ClickCounter() {
  const [count, setCount] = useState(0);

  // Event handler function
  const handleClick = () => {
    setCount(count + 1);
    console.log('Button clicked!');
  };

  return (
    <div className="container">
      <h2>Click Count: {count}</h2>
      <button onClick={handleClick}>Click Me</button>
    </div>
  );
}
```

KEY TAKEAWAY Handling user events is essential for creating interactive applications. React's event system is simple, consistent, and powerful.

HANDLING USER EVENTS (2 OF 2)

Event handler basics, passing data to handlers, preventing default behavior, and where common events are used across form elements.

PASS THE REFERENCE, NOT THE CALL

```
// Correct -- pass the reference
<button onClick={handleClick}>
  Click Me
</button>
```

```
// Incorrect -- calls immediately during render
<button onClick={handleClick()}>
  Click Me
</button>
// this runs on every render, not on click
```

COMMON EVENTS BY ELEMENT

Element	Common Events	Purpose
Button	onClick	Handle button clicks
Input	onChange, onFocus, onBlur	Handle input changes, focus, blur
Form	onSubmit	Handle form submission

PASSING DATA TO EVENT HANDLERS

```
const handleDelete = (id) => {
  console.log('Delete item:', id);
};

<button onClick={() => handleDelete(42)}>Delete</button>
```

PREVENTING DEFAULT BEHAVIOR

```
const handleSubmit = (e) => {
  e.preventDefault(); // stop page refresh
  console.log('Form submitted!');
};

<form onSubmit={handleSubmit}>
  <button type="submit">Submit</button>
</form>
```

KEY TAKEAWAY Always pass the function reference, not the function call; use an arrow function to pass arguments; call `preventDefault()` to stop the default browser behavior.

FORMS AND FORM STATE (1 OF 2)

Forms let users enter data and interact with your application. In React, form elements are controlled using component state, so the UI always reflects the latest values.

WHY USE CONTROLLED COMPONENTS?

- **Single Source of Truth** — state, not the DOM, always holds the current input value.
- **Validation & Error Handling** — every keystroke can be checked before submit.
- **Easier to Manage and Test** — form values are plain data, not DOM queries.
- **UI Always in Sync** — the displayed value always matches component state.

KEY PRINCIPLES

- **Use value to bind** — an input's value prop always comes from state, never from the DOM.
- **Use onChange to update** — every keystroke updates state through the setter.
- **Give every input a name** — so one handler can identify which field changed.
- **Call preventDefault() on submit** — to stop the page from reloading.

HOW FORMS WORK IN REACT



EXAMPLE -- CONTROLLED FORM

```
function UserForm() {
  const [form, setForm] = useState({ name: '', email: '' });

  const handleChange = (e) => {
    const { name, value } = e.target;
    setForm({ ...form, [name]: value });
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log('Submitted:', form);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" value={form.name} onChange={handleChange} />
      <input name="email" value={form.email} onChange={handleChange} />
      <button type="submit">Submit</button>
    </form>
  );
}
```

KEY TAKEAWAY Controlled components keep form state as the single source of truth -- bind value to state, update it via onChange, and prevent default on submit.

FORMS AND FORM STATE (2 OF 2)

A quick reference for common input types, the single-state-object pattern for multi-field forms, and best practices for managing form state.

Input Type	Example	Handling
text	John Doe	value + onChange
email	john@example.com	value + onChange
password		value + onChange (masked)
number	25	value + onChange (numeric)
checkbox	☒ I agree	checked + onChange
radio	☒ Male ☐ Female	checked + onChange
select	Canada ▾	value + onChange

BEST PRACTICES

Keep form state minimal

Provide instant feedback

Validate on change or submit

Reset after successful submission

COMMON PATTERN: SINGLE STATE OBJECT

```
const [form, setForm] = useState({
  name: '',
  email: '',
  password: '',
});
```

IMPORTANT TIPS

- Always use the name attribute for each input.
- Use onChange to keep state in sync with user input.
- Avoid using refs for form state (except uncontrolled forms).
- Validate before submitting the form.

KEY TAKEAWAY Keep state as the single source of truth: prefer one state object for related fields, and validate user input before submitting.

PARENT-TO-CHILD COMMUNICATION

In React, data flows down from parent components to their children using props -- a one-way, predictable data flow.

HOW IT WORKS



KEY TAKEAWAYS

- Parents own the state and pass it to children using props.
- Props are read-only in the child -- a child cannot modify them.
- Data flows down only, one way, keeping the pattern predictable.
- This keeps components simple, reusable, and easy to debug.

EXAMPLE -- APP.TSX PASSES NAME TO GREETING

```
function App() {
  const [name, setName] = useState('Aman');
  return (
    <div>
      <input value={name} onChange={(e) => setName(e.target.value)} />
      <Greeting name={name} />
    </div>
  );
}

function Greeting({ name }) {
  return <h3>Hello, {name}!</h3>; // read-only prop
}
```

Prop Type	Example
String	<Greeting name="Aman" />
Number	<Counter count={10} />
Object / Array	<Customer data={record} />
Function	<Button onClick={handleClick} />

KEY TAKEAWAY Props are read-only and flow one way -- from parent to child. Pass only what a child needs, and keep components focused.

CHILD-TO-PARENT COMMUNICATION

In React, data flows up from child components to their parents using callback props -- children call a function passed down by the parent to send data up.

HOW IT WORKS



KEY TAKEAWAYS

- Children cannot change the parent's state directly.
- Instead, they call a function (callback) passed down as a prop.
- Data flows up; the parent decides what to do with it.
- This keeps components decoupled, reusable, and testable.

EXAMPLE -- CHILD SENDS A MESSAGE TO APP

```
function App() {
  const [message, setMessage] = useState('');
  const handleMessage = (msg) => setMessage(msg);
  return <Child onSendMessage={handleMessage} />;
}

function Child({ onSendMessage }) {
  const [text, setText] = useState('');
  const handleClick = () => {
    onSendMessage(text); // send data up
    setText('');
  };
  return <button onClick={handleClick}>Send to Parent</button>;
}
```

COMMON USE CASES

Forms: input -> parent state

Lists: action -> parent updates list

Filters: selection -> parent applies filter

KEY TAKEAWAY Child-to-parent communication uses callback props: the parent defines a function, passes it down, and the child calls it to send data back up.

STATE UPDATE WORKFLOW

When state is updated, React goes through a well-defined, asynchronous workflow: request the change, React updates and re-renders, and the UI reflects the new state.



WHAT HAPPENS BEHIND THE SCENES

- React does not update the DOM immediately when a setter is called.
- It batches the update and applies it after the current event handler finishes.
- Then React re-renders the component with the new state.
- Finally, React updates only the parts of the real DOM that actually changed.

EXAMPLE -- COUNTER.TSX

```
const [count, setCount] = useState(0);
const handleIncrement = () => {
  setCount(count + 1); // request state update
};
```

BEST PRACTICE -- FUNCTIONAL UPDATES

```
setCount((prev) => prev + 1); // always up to date
```

KEY TAKEAWAY State updates are asynchronous and batched -- use functional updates (`prev => prev + 1`) whenever the new state depends on the previous state.

MANAGING COMPLEX USER INTERFACES (1 OF 2)

As applications grow, UIs become complex with many interactive parts and shared state. React provides patterns to organize state and stay maintainable.

CHALLENGES IN COMPLEX UIs

- Many components and interactions to coordinate.
- Shared state needed across different parts of the UI.
- Deeply nested component trees.
- Managing forms, lists, filters, and pagination together.
- Keeping the UI fast and responsive as it grows.

KEY IDEA

Break the UI into smaller, reusable pieces, manage state effectively, and ensure data flows in the right direction.

COMPONENT DECOMPOSITION EXAMPLE

Dashboard (Page) -> Sidebar, Header, Main Content, User Profile -> Main Content splits into Stats Cards, Chart, Recent Activity. Break the interface into reusable, manageable components.

STRATEGIES FOR MANAGING COMPLEX UIs

Strategy	What It Means
Component Decomposition	Break the UI into small, focused components with a single responsibility.
Lift State Up	Store shared state in the nearest common parent and pass it down via props.
Custom Hooks	Extract and reuse stateful logic across multiple components.
Context API	Share state across deeply nested components without prop drilling.
State Management Libraries	Use tools like Redux Toolkit, Zustand, or Recoil for large-scale needs.
Performance Optimization	Use memoization, lazy loading, and virtualization for better performance.

KEY TAKEAWAY Break the UI into smaller, reusable pieces, manage state effectively, and ensure data flows in the right direction.

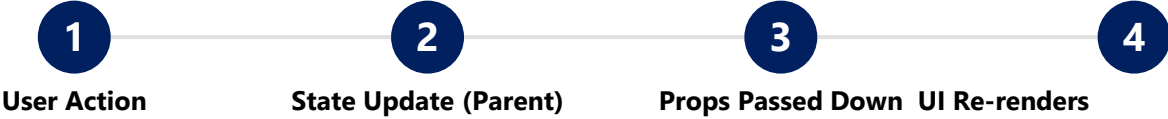
MANAGING COMPLEX USER INTERFACES (2 OF 2)

Choosing the right pattern for the job, and the unidirectional data-flow discipline that keeps large React applications predictable.

WHEN TO USE WHAT?

Need	Recommended Approach
Share state between nearby components	Lift state up
Share state across a deep component tree	Context API
Reuse stateful logic across components	Custom Hooks
Manage complex, app-wide global state	State management library
Improve rendering performance	Memoization, code splitting, virtualization

DATA FLOW IN COMPLEX UIs



Unidirectional data flow keeps your UI predictable and easier to debug -- state is updated in one place and flows down through props.

BEST PRACTICES

- Keep components small & focused
- Keep state close to where it's used
- Derive data instead of duplicating
- Test interactions thoroughly

KEY TAKEAWAY Managing complex UIs is about structuring components, managing state wisely, and optimizing performance -- keep data flow unidirectional and predictable.

REACT STATE BEST PRACTICES

Following these best practices helps you write predictable, maintainable, and performant React applications. Golden rule: keep state minimal and close to where it's used.

#	Practice	What It Means
1	Keep State Local When Possible	Store state at the lowest common ancestor to reduce complexity and unnecessary re-renders.
2	Avoid Redundant or Derived State	Don't store data in state that can be calculated from props or other state -- derive it during render.
3	Use State Updates Immutably	Never mutate state directly; always use the setter and return new values (spread, map, filter).
4	Use Functional Updates When Next State Depends on Previous	Pass a function to the setter to get the latest value and avoid stale-closure bugs.
5	Don't Over-Lift State	Avoid lifting state higher in the tree than necessary -- it makes state harder to track and reuse.
6	Keep State Shape Simple and Flat	Prefer simple structures over deeply nested objects to simplify updates.
7	Avoid Large or Complex State	Split large state into smaller, focused pieces to improve readability and performance.
8	Clean Up State When No Longer Needed	Remove obsolete state and reset it on unmount if needed to prevent memory leaks.

Do	Don't
Use meaningful state names. Keep components focused. Use controlled components for forms. Use <code>useReducer</code> or <code>Context</code> for complex state logic.	Don't mutate state directly. Don't store non-serializable values in state unless necessary. Don't duplicate data in multiple places. Don't ignore React warnings about state updates.

KEY TAKEAWAY Golden Rule: keep state minimal, meaningful, and close to where it is used. When in doubt, simplify.

LAB OVERVIEW -- STATE AND EVENT MANAGEMENT

Lab 34: React State and Event Management -- lifting the CRM dashboard's state into App (lab34-crm). Theory 1 Hour / Lab 2 Hours / Total 3 Hours.

BUSINESS SCENARIO

- Leadership freezes the gate: no CRM page state ships without immutable updates, derived filters, exclusive form modes, and interaction tests covering create/edit/cancel/search.
- You own that contract for Amina (CUS-1001, ACTIVE), Ravi (CUS-1002, PROSPECT), search "amina", and validation failures on a blank name or a bad email.
- This lifts Lab 33's presentation-only components (CustomerList, CustomerForm, CustomerToolbar) into one App with useState as the single source of truth, ahead of Lab 35's live API wiring.

LEARNING OBJECTIVES

- Store customer records with useState as the single source of truth.
- Create controlled search and form inputs.
- Derive filtered results during render -- no duplicate filtered state.
- Lift selection and form mode into the page with a discriminated union.
- Write immutable create and update handlers.
- Synchronize document.title with useEffect, with cleanup.

WHAT YOU BUILD

- Copy lab33-crm to lab34-crm; lift customers/query/mode/draft/errors into App; control search; derive visible; model mode as closed/create/edit; validate before save; append/replace customers immutably; sync document.title with useEffect; write ≥ 8 RTL flow tests for create/edit/cancel/search.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor every example; correlation id lab-request-001 appears in create/update/cancel logs.

LAB TASKS AND DELIVERABLES

lab34-crm -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Copy lab33-crm to lab34-crm; lift customers and query into App with useState.
2	Control the search input (value + onChange) wired to CustomerToolbar.
3	Derive visible customers by filtering during render -- never a second useState.
4	Model form mode as a discriminated union: closed / create / edit.
5	Update controlled draft fields immutably; clear only the edited field's error.
6	Validate before saving; block blank name / invalid email with field errors.
7	Append and replace customers immutably (spread) -- never push or mutate in place.
8	Sync document.title with useEffect (with cleanup); write ≥8 RTL flow tests.

EXPECTED DELIVERABLES

- lab34-crm/crm-ui with lifted state CRUD + search.
- Discriminated form modes; immutable updates.
- Client validation with accessible errors.
- Title useEffect with cleanup; no derived-state effects.
- ≥8 RTL interaction tests + green build (twice).
- State notes (docs/state-notes.md) + evidence screenshots.

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Prod Awareness 10 · Docs 10

KEY TAKEAWAY In-place mutations or filtered-state effects lose core marks; happy-path-only tests lose automated-verification marks -- prove create, edit, cancel, and search all work.

MODULE 34 SUMMARY

In this module, we learned how to manage state and events in React to build dynamic, interactive, and responsive user interfaces.

KEY CONCEPTS COVERED



Local State (useState)

useState adds state to a functional component; state is local, private, and triggers re-renders.



Handle Events

Respond to user interactions with onClick, onChange, and onSubmit event handlers.



Controlled Forms

Bind value to state and update via onChange, using one state object for related fields.



Lift State Up

Share data via props (down) and callback functions (up); keep data flow unidirectional.



Side Effects (useEffect)

Synchronize with the outside world -- fetches, subscriptions, timers -- and always clean up.



Best Practices

Update state immutably, derive instead of duplicating, and keep state minimal and local.

MODULE FLOW RECAP

1

useState

2

Event Handlers

3

Controlled Forms

4

Lifted State

5

useEffect

6

Lab: lab34-crm

KEY TAKEAWAY Effective state and event management is the foundation of every interactive React application -- state drives the UI, events drive state, and immutability keeps everything predictable.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of React state fundamentals, hooks, re-renders, and controlled inputs.

1. What is the primary purpose of state in React?

- A. To store data that never changes
- B. To manage dynamic data that affects the UI**
- C. To handle routing between pages
- D. To style components dynamically

3. What happens when state is updated in a React component?

- A. The DOM is updated immediately
- B. The component re-renders with the new state**
- C. The page reloads
- D. React ignores the update

2. Which Hook is used to add state to a functional component?

- A. useEffect
- B. useState**
- C. useReducer
- D. useContext

4. Which event is typically used to handle changes in a text input?

- A. onClick
- B. onChange**
- C. onSubmit
- D. onLoad

KEY TAKEAWAY State manages data that changes over time and drives the UI; useState adds it to a component; updating it re-renders; onChange handles text input changes.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on parent-child data flow, callbacks, functional state updates, and preventing unnecessary re-renders.

5. How do you pass data from a parent component to a child component?

- A. Using state
- B. Using props**
- C. Using context only
- D. Using localStorage

7. What is the recommended way to update state based on the previous state?

- A. `setCount(count + 1)`
- B. `setCount(prev => prev + 1)`**
- C. `setCount({ count: count + 1 })`
- D. `setCount(prev.count + 1)`

6. How can a child component send data back to its parent?

- A. Using props
- B. Using a callback function passed via props**
- C. Using global variables
- D. Using local state

8. Which of the following helps prevent unnecessary re-renders of components?

- A. `useState`
- B. `React.memo`**
- C. `useEffect`
- D. `setState`

KEY TAKEAWAY Props pass data down; a callback prop sends data up; prefer functional updates (`prev => prev + 1`); `React.memo` helps prevent unnecessary re-renders.

MODULE 34 COMPLETE

State and Event Management in React

You can now manage local and lifted state with `useState`, handle user events and controlled forms, synchronize side effects with `useEffect`, and update state immutably.